FUNCIONES PROPIAS

10145 - FUNDAMENTOS DE PROGRAMACIÓN PARA INGENIERÍA

10110 – FUNDAMENTOS DE COMPUTACIÓN Y PROGRAMACIÓN

RESUMEN DE CONTENIDOS

SINTAXIS DE FUNCIONES PROPIAS

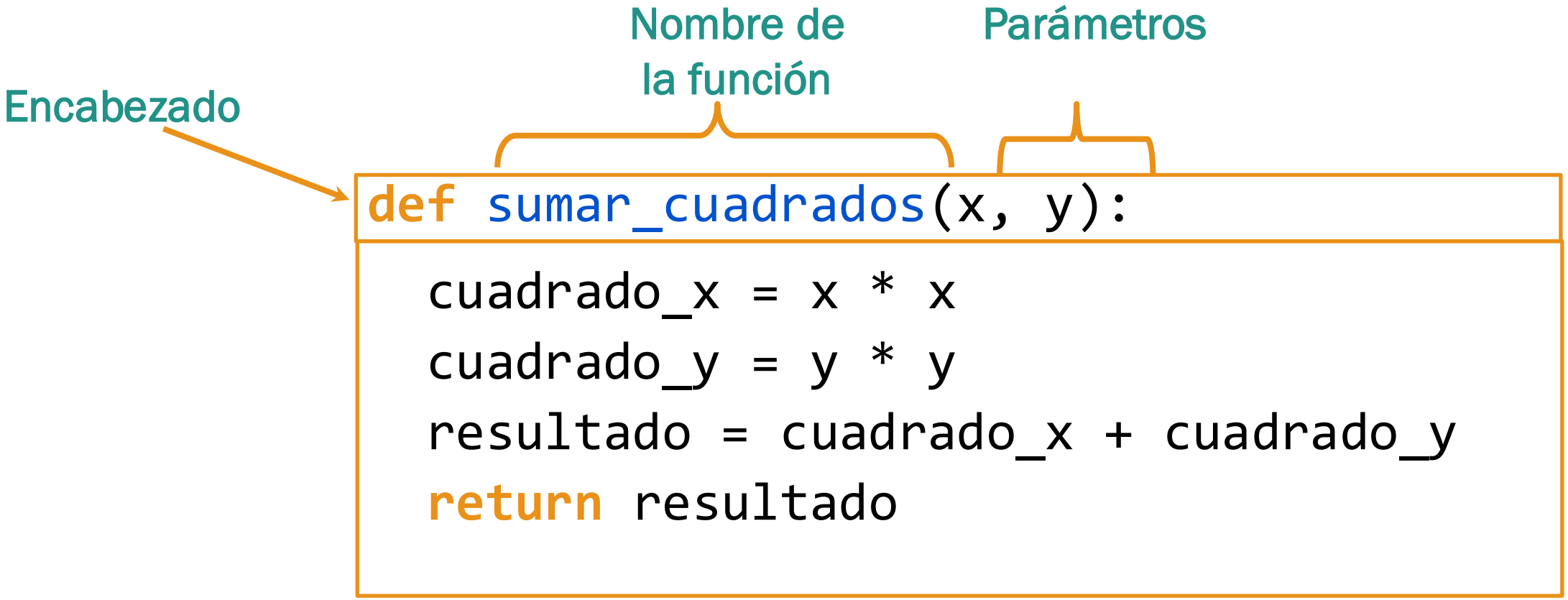
- La sintaxis para definir una función propia en Python es:

```
def sumar_cuadrados(x, y):
    cuadrado_x = x * x
    cuadrado_y = y * y
    resultado = cuadrado_x + cuadrado_y
    return resultado
```

SINTAXIS DE FUNCIONES PROPIAS

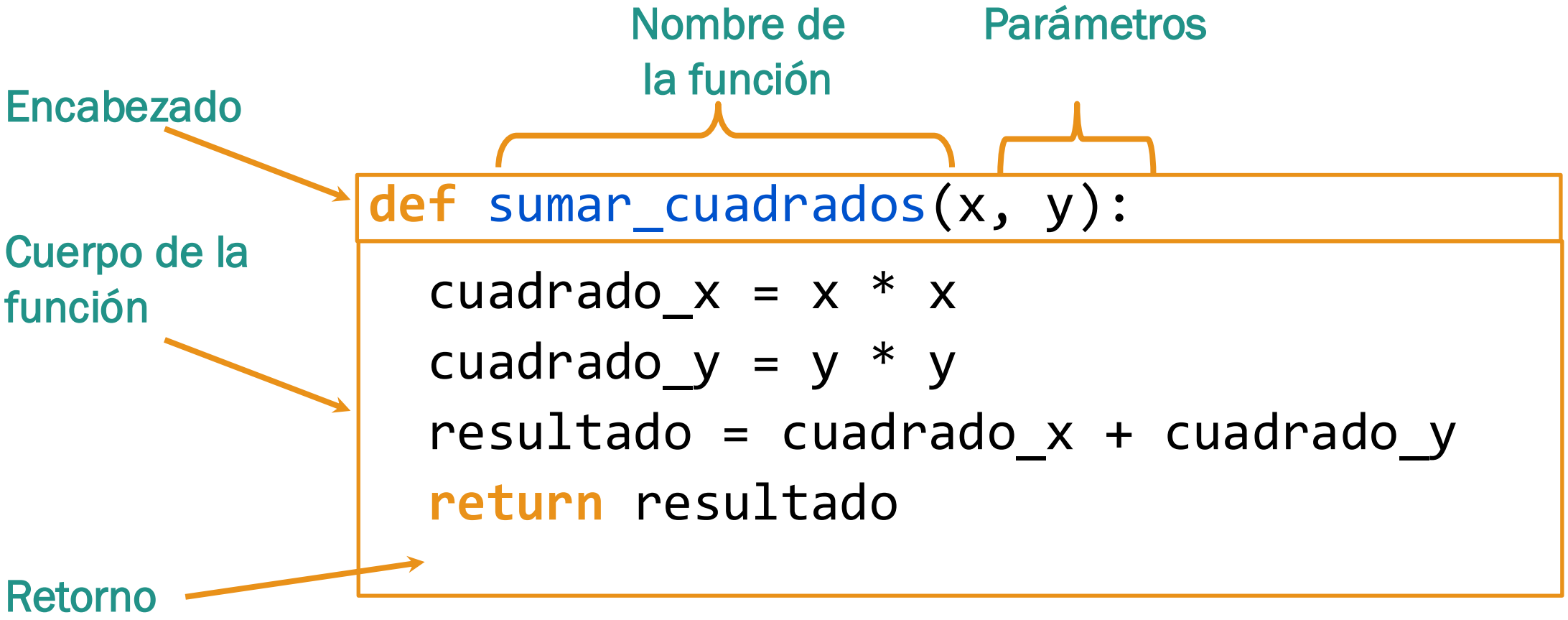
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA
UNIVERSIDAD DE SANTIAGO DE CHILE

- La sintaxis para definir una función propia en Python es:



SINTAXIS DE FUNCIONES PROPIAS

- La sintaxis para definir una función propia en Python es:



SINTAXIS DE FUNCIONES PROPIAS

- La sintaxis para definir una función propia en Python es:

```
def sumar_cuadrados(x, y):
    cuadrado_x = x * x
    cuadrado_y = y * y
    resultado = cuadrado_x + cuadrado_y
    return resultado

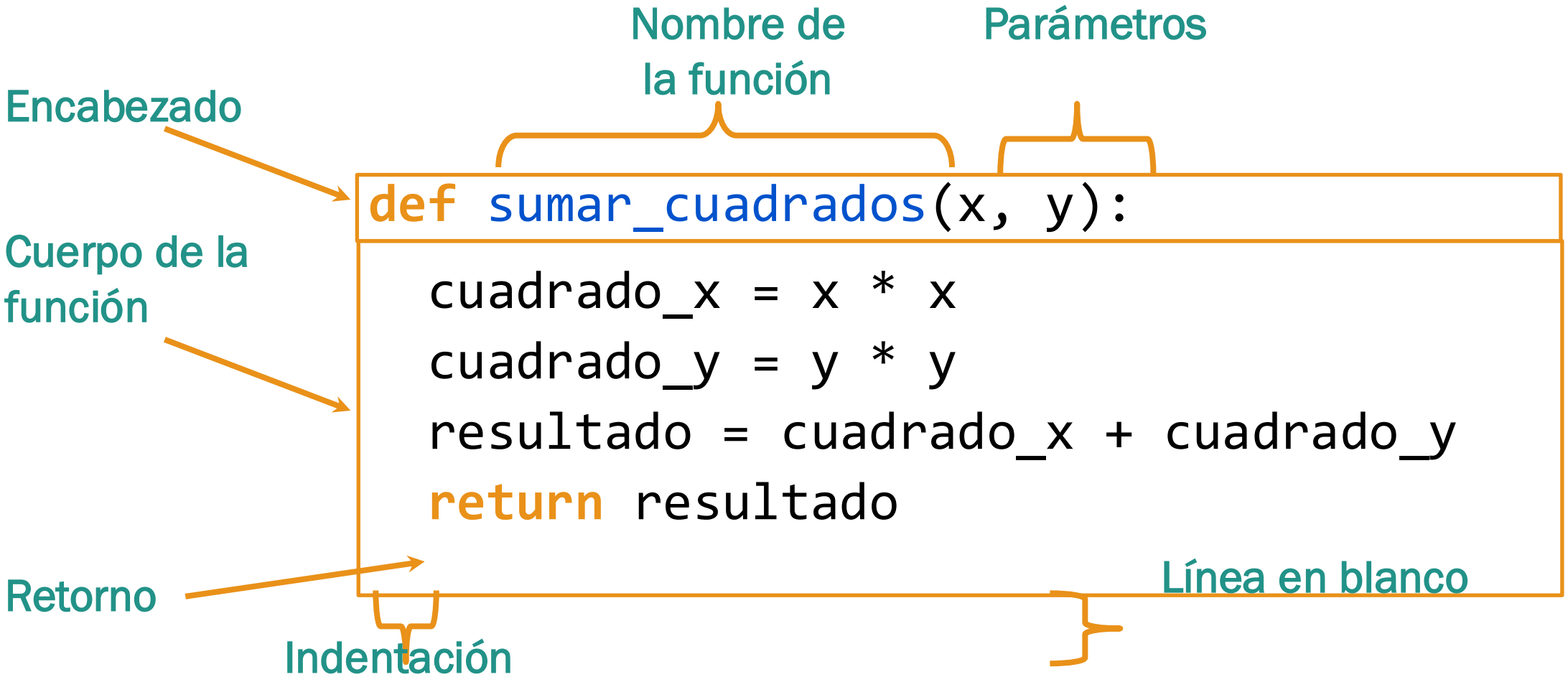
```

Indentación

Línea en blanco

SINTAXIS DE FUNCIONES PROPIAS

- La sintaxis para definir una función propia en Python es:

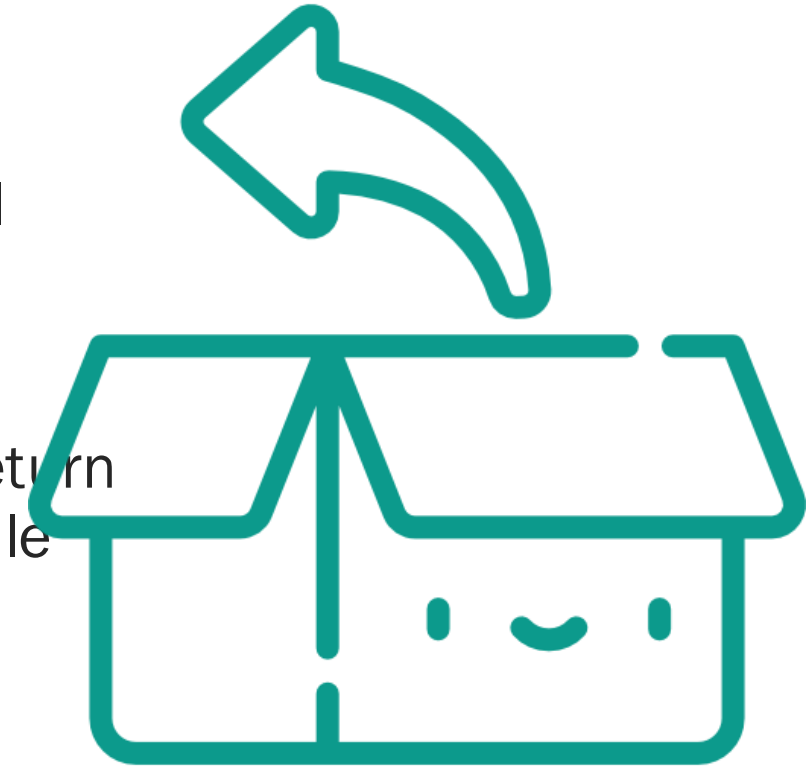


SENTENCIA `return`

- A diferencia de un `print()`, que **entrega un valor sólo para su visualización**, el `return` entrega **un valor que podemos manipular y operar** dentro de un programa

- El uso del `return` debe ser una decisión estratégica del programador, pues puede ayudar a **optimizar el código**

- En las próximas clases aprenderemos cómo utilizar el return y funciones propias para hacer repeticiones sin usar while



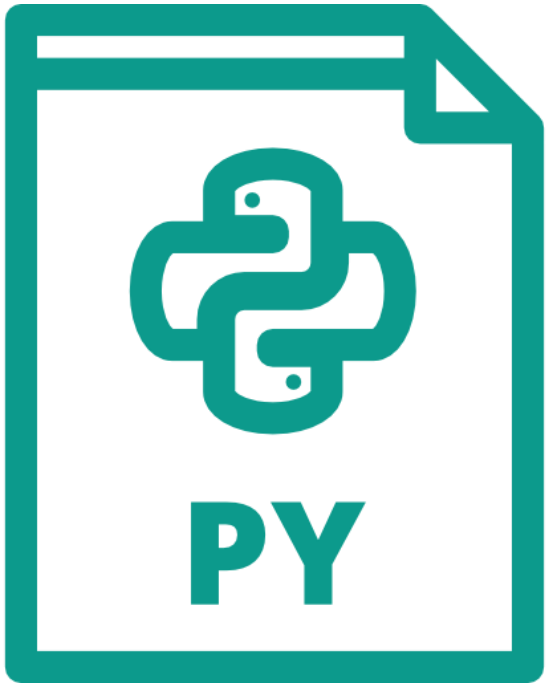
Autor: freepik
Fuente: www.flaticon.com

COMENTARIOS

- Cada función propia que se defina debe tener al inicio, en sus comentarios:
  - Qué es lo que la función hace
  - Cuáles son las entradas que requiere, indicando tipo de dato para cada entrada
  - Cuál es la salida que entrega y el tipo de dato de esta salida
  - Todo esto en un comentario multilínea *(docstring)* luego del encabezado de la función
- Para ejemplificar, utilizaremos como ejemplo la función `es_primo()`

COMENTARIOS

```python
def es_primo(numero):
    """Función que determina si un número entero
    positivo es primo o no.
    Entrada: número (int) mayor o igual a 2.
    Salida: booleano (True si el número es primo)
             (False si el número no es primo)
    """
    i = 2
    while i < numero:
        if numero % i == 0:
            return False
        i = i + 1
    return True
```



Autor: Freepik
Fuente: www.flaticon.com

Un programa se organiza en dos bloques:

- **Bloque de definiciones**: carga las herramientas que el programa necesita; no pide datos ni imprime nada
  - Importación de módulos o funciones
  - Definición de constantes, con el nombre en mayúsculas
  - Definición de las funciones propias
- **Bloque principal**: usa esas herramientas con valores concretos
  - **Entrada**: los datos que llegan de fuera, con `input()`
  - **Procesamiento**: los cálculos, invocando las funciones definidas
  - **Salida**: la entrega de resultados, con `print()`

- **Parámetros**: los nombres de las entradas al definir la función; viven en la línea del `def`
- **Argumentos**: los valores concretos que se entregan al invocarla

```
def calcular_trabajo(fuerza, distancia, angulo):
  return fuerza * distancia * math.cos(angulo)

trabajo = calcular_trabajo(10, 5, math.pi / 4)
```

  - `fuerza`, `distancia` y `angulo` son los parámetros
  - `10`, `5` y `math.pi / 4` son los argumentos

VISIBILIDAD

- Cuando creamos una variable, esta tiene una propiedad llamada visibilidad (scope)

- Técnicamente, a esta visibilidad se le llama **alcance** y existen esencialmente dos tipos de alcances:
  - las variables locales, que son aquellas que sólo existen dentro de la función que la crea
  - las **variables globales** son visibles por todas las sentencias que siguen a su creación, excepto donde existe una variable local con el mismo nombre

- El factor que determina si una variable es local o global, es el **dónde es creada**, a pesar de que existen métodos para definirlo explícitamente



Autor: Freepik
Fuente: www.flaticon.com

MALAS PRÁCTICAS CON FUNCIONES

Mala práctica del curso: Python permite modificar desde dentro de una función una variable definida fuera; aquí no lo haremos.

- **Consultar** una variable global sin cambiarla es aceptable: es lo que hacemos con `math.pi`
- **Modificarla** desde adentro es lo que se desaconseja

Una función que sólo recibe parámetros y devuelve un resultado se entiende leyéndola sola: sus entradas están en el `def` y su salida en el `return`.

Si necesita un dato de afuera, se le pasa como parámetro; si produce un resultado, lo entrega con `return`.

DIVISION EN SUBPROBLEMAS

Separar un problema en funciones más pequeñas lo vuelve más simple.
Calcular una potencia usando sólo sumas:

```
def multiplicacion(a, b):
  i = 0
  resultado = 0
  while i < b:
    resultado = a + resultado
    i = i + 1
  return resultado

def exponenciacion(base, exponente):
  i = 0
  potencia = 1
  while i < exponente:
    potencia = multiplicacion(potencia, base)
    i = i + 1
  return potencia
```

ARGUMENTOS

- **Posicionales**: se asignan por el orden de la invocación y son obligatorios

```
def calcular_division(dividendo, divisor):
  return dividendo / divisor
calcular_division(10, 2)  entrega 5.0
calcular_division(2, 10)  entrega 0.2
```

- **Con valor por defecto**: opcionales, van después de los posicionales

```
def calcular_promedio(nota_1, nota_2, nota_3, decimales=1):
  return round((nota_1 + nota_2 + nota_3) / 3, decimales)
calcular_promedio(5.4, 3.8, 6.7)    usa 1 decimal
calcular_promedio(5.4, 3.8, 6.7, 3)  usa 3 decimales
```

ARGUMENTOS

- Si se conocen los nombres de los parámetros, los argumentos se pueden entregar indicando a cuál corresponde cada uno

  ```
  calcular_promedio(nota_2=3.8, decimales=2, nota_1=5.4, nota_3=6.7)
  ```

  el **orden deja de importar**

  - Gana en claridad cuando la función tiene muchos parámetros, pero exige conocer sus nombres exactos
  - Regla del curso: usa el nombre cuando ayude a entender qué significa cada valor, y el orden cuando la función sea corta y los parámetros evidentes

  - El = de un parámetro va sin espacios, en la definición y en la invocación: es la única situación en que no los lleva